Enterprise VOIP / Softphone Endpoint Support & Troubleshooting

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

What it delivers . . . 4

Why it matters . . . 4

Bottom line . . . 4

Solution Overview 4

The softphone / VOIP endpoint stack . . . 5

What the endpoint owns vs. what the telephony platform owns . . . 5

The root-cause tree (how to think about a call-quality ticket) . . . 6

The repeatable method . . . 7

Requirements 7

R1. Endpoint access (local + remote) . . . 7

R2. Audio-device inventory . . . 7

R3. Network / QoS visibility . . . 8

R4. Softphone client details . . . 8

R5. Baseline "known-good" call state . . . 8

Diagnostic Playbook 9

Symptom A — No audio (can't hear, can't be heard) or one-way audio . . . 9

Symptom B — Echo, robotic, choppy, or garbled audio . . . 10

Symptom C — Wrong default device (calls keep using the laptop mic/speaker) . . . 10

Symptom D — Headset not recognized / not appearing as a device . . . 11

Symptom E — Jitter / latency / dropped calls (quality degrades or calls drop) . . . 11

Symptom F — Softphone client crashes, hangs, or won't launch / no audio in client . . . 12

Closing every ticket . . . 13

Testing 13

Gate 1 — Correct audio device . . . 13

Gate 2 — Softphone client healthy . . . 13

Gate 3 — Network within thresholds . . . 14

Overall result . . . 14

Pilot validation . . . 15

Troubleshooting 15

VOIPSUP-Discovery.ps1 . . . 15

VOIPSUP-Validation.ps1 / VOIPSUP-Report.ps1 . . . 15

VOIPSUP-Remediation.ps1 . . . 16

VOIPSUP-Set-ScopeTargeting.ps1 . . . 16

General . . . 16

Best Practices 16
Work the tree top-down, every time . . . 17
Define "correct" before you judge "wrong" . . . 17
Measure the network; never assume it . . . 17
Keep the three surfaces coherent . . . 17
Prefer wired; treat Wi-Fi and VPN as suspects . . . 17
Remediate safely: -WhatIf first, then execute . . . 17
Pilot before fleet-wide . . . 17
Respect the escalation boundary . . . 17
Evidence-based escalation to the telephony team . . . 18
Deliverables Summary 18
Documents . . . 18
Scripts . . . 19
How they fit the method . . . 20
Outcome . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary

When a call drops, echoes, or a headset won't work, the ticket lands on desktop support — the team that owns the **endpoint**, not the phone system. Without a method, those tickets bounce: support guesses, then punts to the telephony team, who bounce it back because the endpoint was never actually verified. The result is slow resolution, frustrated users, and wasted specialist time.

This solution gives desktop support a **root-cause method and five PowerShell scripts** for the endpoint side of enterprise VOIP/softphone — audio-device selection, headset and driver health, network and QoS conditions, and softphone client health — so the team **resolves what it controls and escalates the rest with hard evidence.**

What it delivers

- **A clear ownership boundary.** The softphone/VOIP endpoint stack (audio device → client → network/QoS

→ service), spelling out what the endpoint owns versus what belongs to the telephony platform — so tickets go to the right team the first time.

- **A symptom-driven diagnostic playbook.** No/one-way audio, echo/robotic, wrong default device, headset

not recognized, jitter/latency, and client crashes — each with exact, click-by-click Windows and Intune checks and a stop-at-the-first-broken-link discipline.

- **Five PowerShell scripts.** Audio-device + client discovery, per-device audio/network validation, a

VOIP-readiness/quality report, safe device/driver remediation (defaults to preview-only), and a pilot target-group builder.

- **Pass/fail gates and evidence-based escalation.** Correct device, healthy client, network within

thresholds — with an escalation template that hands the telephony team verified data, not a hunch.

Why it matters

Call-quality tickets are high-visibility and high-frustration. This toolkit turns an ad-hoc, ticket- bouncing process into a **repeatable method your team keeps and reuses** — faster resolution of the issues desktop support can actually control, and clean, data-backed escalations for the ones it can't. It is white-label, vendor-neutral about the telephony platform, and precise on the Windows/endpoint side.

Bottom line

Call-quality and softphone tickets resolved at the endpoint — or escalated with hard evidence — using a repeatable method and scripts your support team keeps and reuses, at a fraction of the cost of custom consulting.

Solution Overview

When a call drops, echoes, or a headset refuses to work, the ticket almost always lands on desktop support — the team that owns the **endpoint**, not the telephony platform. This solution gives you a repeatable, root-cause method (and five PowerShell scripts) for the **endpoint side** of enterprise VOIP and softphone: audio-device selection, headset and driver health, network and QoS conditions on the device, and the softphone client's own health. The goal is simple: **resolve what desktop support controls, and escalate the rest to the telephony team with hard evidence** instead of a hunch.

This document is deliberately **vendor-neutral about the telephony platform and softphone client**. The exact console screens, dial-plan behavior, and server-side call routing of any given platform are that vendor's domain — defer those internals to the vendor's official documentation. What this solution authors at precise, click-by-click detail is the **Windows / Intune / endpoint side**, which is what you actually own.

The softphone / VOIP endpoint stack

Think of a call on a managed Windows endpoint as a chain. A problem at any link degrades or breaks the call, and each link has a different owner:

1. **Audio device** — the headset, speakerphone, dock, or built-in mic/speakers. This is where sound physically enters and leaves. Covers the physical connection (USB, Bluetooth, 3.5 mm), the device driver, and the firmware on the headset itself.
1. **Windows audio subsystem** — which device is selected as the **Default Communication Device**, per-app routing, sample rate, exclusive-mode settings, volume/mute state, and the enhancements (noise suppression, echo cancellation) Windows or the driver layers on.
1. **Softphone client** — the application that turns audio into VOIP packets: its version, its own audio-device selection (which can differ from the Windows default), its microphone/speaker permissions, its logs, and whether the process is healthy or crashing.
1. **Network / QoS (endpoint view)** — the local network path the call takes: Wi-Fi vs wired, adapter health, jitter, latency, packet loss to the media edge, VPN interference, and whether QoS/DSCP marking is actually being applied on the endpoint.
1. **Telephony service (platform side)** — the VOIP platform's servers, media relays, dial plan, and carrier trunks. **This link is owned by the telephony vendor/team, not desktop support.**

Desktop support owns links **1 through 4**. Link **5** is the escalation boundary.

What the endpoint owns vs. what the telephony platform owns

Concern	Endpoint owns (this solution)	Telephony platform owns (escalate)
Which mic/speaker is used, and its driver	Yes	—
Headset firmware / USB / Bluetooth pairing	Yes	—
Windows Default Communication Device	Yes	—

Concern	Endpoint owns (this solution)	Telephony platform owns (escalate)
Client version, install health, local permissions	Yes	—
Local jitter/latency/packet-loss to the media edge	Yes (measure & report)	—
QoS/DSCP marking applied on the device	Yes (configure/verify)	Enforcement across WAN/LAN switches
Dial plan, call routing, number provisioning	—	Yes
Media-relay / SBC / carrier-trunk health	—	Yes
Platform-side codec negotiation and server config	—	Yes
Account/licensing on the platform	—	Yes

The line to remember: **if it lives on the Windows device or the network path leaving it, it's yours; if it lives on the platform's servers or the carrier, it escalates.**

The root-cause tree (how to think about a call-quality ticket)

Every endpoint-side call-quality investigation follows the same top-down isolation:

```
Call-quality symptom reported
        |
        v
[1] Is the RIGHT audio device selected & healthy?
        |  (correct Default Communication Device? headset recognized? driver OK?)
        |-- NO  --> fix device selection / driver / connection  (ENDPOINT)
        |
        v  YES
[2] Is the SOFTPHONE CLIENT healthy?
        |  (current version? not crashing? using the intended device? permissions granted?)
        |-- NO  --> repair/update/reset client, grant mic permission  (ENDPOINT)
        |
        v  YES
[3] Is the NETWORK / QoS within thresholds?
        |  (jitter/latency/loss OK? wired vs Wi-Fi? VPN? DSCP marked?)
        |-- NO  --> move to wired / fix adapter / apply QoS  (ENDPOINT, partly)
        |            residual WAN/LAN QoS enforcement --> ESCALATE with data
        |
        v  YES  (device good, client good, network good)
[4] The endpoint is clean --> ESCALATE to telephony team WITH EVIDENCE
             (device report, client version, jitter/latency/loss numbers,
              timestamps, and what you already ruled out)
```

The discipline is to **walk the tree top-down and stop at the first broken link** — never guess at the network when the wrong microphone is selected, and never bounce a ticket to telephony until links 1–4 are provably clean. The scripts in this solution automate the checks at each level: **Discovery** inventories the audio devices and client, **Validation** tests the selected device and the local network, **Report** produces a VOIP-readiness/quality summary, **Remediation** applies safe endpoint fixes, and **Set-ScopeTargeting** builds the pilot group so you validate on a controlled set of devices before fleet-wide use.

The repeatable method

Assess → Configure → Pilot → Validate → Expand. Assess the endpoint with **Discovery**; configure the correct device/client/QoS state and remediate safely with **Remediation**; pilot on a controlled group built with **Set-ScopeTargeting**; validate against the pass/fail gates with **Validation** and **Report**; then expand the same checks fleet-wide. Escalation to the telephony team is not a failure of the method — it is the correct outcome when the endpoint is clean and the evidence points at the platform.

Requirements

Before you troubleshoot the endpoint side of a VOIP/softphone issue, confirm the prerequisites below. The category-level prerequisites (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, Windows baseline, network access) are covered once in **global-requirements-infrastructure.md** — confirm those first. The items here are **specific to VOIP/softphone endpoint support**. Each entry follows the same pattern: **Description / Why / Verify / Implement if missing.**

R1. Endpoint access (local + remote)

Description: The ability to run PowerShell on the affected Windows device — either signed in locally with local-administrator rights, or via a remote-support/Intune remote channel.

Why: Every endpoint check (audio-device state, driver status, client version, network path) executes on the device. Read-only discovery works as a standard user; driver and device remediation needs local admin.

Verify: On the device, open **Windows PowerShell** and run `whoami /groups` — confirm the session shows an administrators membership for remediation, or confirm your remote-support tool can reach the device and open a PowerShell session.

Implement if missing: Grant the technician account local-admin (or use an Intune/LAPS-managed local admin), or configure your remote-support tooling to reach enrolled devices. Read-only discovery and validation can proceed without admin; only remediation requires it.

R2. Audio-device inventory

Description: A known, current list of the audio input and output devices on the endpoint — the built-in mic/speakers plus any headset, speakerphone, dock, or Bluetooth device — and which one is **supposed** to be the default communication device.

Why: The single most common endpoint cause of "no audio," "one-way audio," or "wrong device" is that the softphone (or Windows) is using an unexpected device. You cannot judge "wrong device" without knowing which device is correct.

Verify: Settings → **System** → **Sound** shows the current input and output devices; **Sound Control Panel** → **Recording / Playback** shows every device and which is marked **Default Communication Device**. The **VOIPSUP-Discovery.ps1** script enumerates all audio endpoints and flags the current defaults.

Implement if missing: Run **VOIPSUP-Discovery.ps1** to build the inventory automatically, and record the intended headset model per role/user so "correct device" is defined for each endpoint.

R3. Network / QoS visibility

Description: The ability to measure the local network path from the endpoint — connection type (wired vs Wi-Fi), adapter health, and jitter/latency/packet-loss toward the media edge — and to see whether QoS/DSCP marking is applied on the device.

Why: Echo, robotic/choppy audio, and drop-outs are frequently network conditions (jitter, latency, loss), not device faults. You must measure the endpoint's network path to know whether to fix it locally, apply QoS, or escalate WAN/LAN QoS enforcement to the network/telephony team.

Verify: `Get-NetAdapter` shows adapter link speed and status; `Test-Connection`/`ping` and the **VOIPSUP-Validation.ps1** script report latency and jitter to a target; QoS policy presence is visible under **Local Group Policy** → **Policy-based QoS** or via `Get-NetQosPolicy`.

Implement if missing: Establish a known good media-edge target to test against, ensure ICMP or an approved probe is permitted for measurement, and (if QoS is expected) deploy the QoS policy via Intune or Group Policy so DSCP marking is present on the endpoint.

R4. Softphone client details

Description: Which softphone/VOIP client is deployed, its **expected/minimum supported version**, its install location, its log location, and how it is deployed/updated (Intune app, MSI, per-user).

Why: Client crashes, no-audio-in-client, and codec/quality issues are often stale versions, corrupt installs, missing microphone permission, or the client using a different audio device than Windows. You need the expected version and log path to judge client health and to reset it safely.

Verify: The **VOIPSUP-Discovery.ps1** script reports installed VOIP/softphone client(s) and their versions from the installed-programs inventory; Settings → **Privacy & security** → **Microphone** shows whether the client (and desktop apps generally) may use the mic.

Implement if missing: Document the sanctioned client and its minimum supported version, package it for Intune deployment/update, and confirm microphone privacy permission is allowed for the client. For platform-internal client behavior (in-client call settings, server sign-in), defer to the client vendor's official documentation.

R5. Baseline "known-good" call state

Description: A documented definition of what a healthy call looks like on your endpoints: the correct default communication device per role, the minimum client version, and the network thresholds you consider acceptable (e.g., latency and jitter ceilings, packet-loss ceiling).

Why: Pass/fail gates (see the Testing guide) require thresholds. Without a baseline you cannot say a device "passed" — you can only say it "seems fine," which does not survive escalation.

Verify: The thresholds are written down and used by **VOIPSUP-Validation.ps1** and **VOIPSUP-Report.ps1** (they accept threshold parameters and mark PASS/FAIL against them).

Implement if missing: Set initial thresholds from your platform vendor's published endpoint network guidance (latency, jitter, and loss ceilings), record them, and tune after piloting.

Diagnostic Playbook

This is the working guide desktop support follows when a VOIP/softphone call-quality ticket arrives. It is **symptom-driven**: find the reported symptom, follow that branch, run the exact endpoint-side checks in order, and **stop at the first broken link**. Walk every branch top-down through the root-cause tree (audio device → client → network/QoS → escalate). The Windows/Intune steps are precise; anything that lives on the telephony platform's servers is called out as an escalation, not something you fix here.

Before any branch — set the frame:

1. Run **VOIPSUP-Discovery.ps1** to capture the audio-device inventory and the softphone client version.
2. Ask the user: *when* did it start, *is it every call or some calls*, *which headset*, *wired or Wi-Fi*.
3. Reproduce if you can. A ticket you can reproduce is a ticket you can prove fixed.

Symptom A — No audio (can't hear, can't be heard) or one-way audio

Most likely: wrong device selected, muted/zeroed level, or missing mic permission — not the network.

1. **Check the selected devices.** Settings → **System** → **Sound**. Confirm the correct output (speaker)

and input (microphone) are chosen. Then open **Sound Control Panel** (`mmsys.cpl`) → **Playback** and **Recording** tabs and confirm the headset is set as **Default Communication Device** (the green phone icon), not just Default Device. `VOIPSUP-Discovery.ps1` reports the current defaults so you can compare against the intended device from your inventory (R2).

1. **Check levels and mute.** In **Recording** → **[mic]** → **Properties** → **Levels**, confirm the mic is not

muted and the level is not at 0. In **Playback** → **[speaker]** → **Properties** → **Levels**, same. Watch the green level meter on the Recording tab while the user speaks — no movement means the mic isn't capturing.

1. **Check microphone privacy permission.** Settings → **Privacy & security** → **Microphone** — confirm

Microphone access is On and that **desktop apps** (and the specific client, if listed) may use the mic. A blocked mic permission is a classic "one-way audio" cause.

1. **Check the client's own device selection.** The softphone client can select a *different* device

than Windows. Open the client's audio/device settings and confirm it uses the same headset. (This is client UI — follow the vendor's docs for the exact menu; the point is that the client device must match the Windows default communication device.)

1. **Check the physical connection.** For USB: try a different port (avoid unpowered hubs). For

Bluetooth: confirm paired and connected, and that it's in the correct profile (a headset stuck in music/A2DP mode instead of hands-free/HFP gives you output but no working mic — reconnect or re-select the hands-free device).

1. **Isolate one-way audio.** If the user can hear but not be heard → the **input** side (mic device,

mic level, mic permission, mic mute). If they can be heard but not hear → the **output** side (wrong speaker device, output level/mute). One-way audio is almost always a one-sided endpoint fault.

1. **Still broken with the right device, level, permission, and client selection?** Run

VOIPSUP-Validation.ps1 to confirm the device tests healthy, then escalate: if the endpoint proves the correct device is active and capturing but the platform reports no media, that's a **platform/media-path issue** → **escalate with the device report.**

Remediation shortcut: `VOIPSUP-Remediation.ps1` can set the correct Default Communication Device and restart the Windows audio service. It defaults to `-WhatIf` — review the planned action, then re-run with `-Execute` (or `-WhatIf:$false`) to apply.

Symptom B — Echo, robotic, choppy, or garbled audio

Most likely: network jitter/latency/loss, or a device-enhancement/sample-rate conflict.

1. **Wired vs Wi-Fi first.** `Get-NetAdapter` (or Settings → **Network & internet**) — is the call on

Wi-Fi? Moving to a wired connection is the single highest-yield fix for robotic/choppy audio. If only Wi-Fi is possible, note signal strength and channel congestion.

1. **Measure the network path.** Run **VOIPSUP-Validation.ps1** to report **latency, jitter, and packet

loss** to the media-edge target from R3/R5. Compare against your thresholds. High jitter or loss → robotic/choppy audio; high latency → talk-over/echo perception.

1. **Rule out local congestion.** Check for a large upload/download or backup running during the call

(`Get-Process`, Task Manager → Network). Saturated uplink starves call media.

1. **Check audio enhancements.** **Sound Control Panel → Recording/Playback → [device] → Properties →

Advanced / Enhancements **— a mismatched** sample rate **or aggressive** signal enhancements** can cause robotic audio. Try the default format and, if enhancements are on, test with them off.

1. **Echo specifically.** Echo is usually the *far* end hearing themselves via *your* speaker feeding

your mic. Confirm the user is on a headset (not open speakers + mic), and that echo cancellation is not disabled in the device/client settings. If a headset is in use and it still echoes across multiple calls/parties, that points at platform-side media handling → **escalate**.

1. **VPN interference.** If the endpoint is on VPN, media may be tromboning through a distant gateway —

note it; split-tunnel for VOIP media is a network-team change → **escalate the QoS/VPN routing** with the latency numbers.

1. **Verify QoS marking.** `Get-NetQosPolicy` — confirm DSCP marking exists for the client's media. If

the endpoint isn't marking traffic, deploy the QoS policy (see Remediation); if it *is* marked but quality is still poor, WAN/LAN QoS **enforcement** is the network/telephony team's domain → **escalate with the measured jitter/latency/loss.**

Symptom C — Wrong default device (calls keep using the laptop mic/speaker)

Most likely: Default Communication Device not set to the headset, or Windows switching on reconnect.

1. **Confirm the intended device.** From R2, know which device *should* be default for this user/role.

2. **Set the Default Communication Device. Sound Control Panel** → **Playback** and **Recording** →

right-click the correct headset → **Set as Default Communication Device**. Note that Windows tracks *Default Device* and *Default Communication Device* separately — calls use the communication default.

1. **Handle reconnect churn.** USB/Bluetooth headsets that disconnect (sleep, undock) can lose the

default when they return. `VOIPSUP-Remediation.ps1` can re-assert the correct default communication device (default `-WhatIf`); for persistent churn, standardize on one connection method and keep the headset firmware/driver current (Symptom D).

1. **Client override.** Even with Windows correct, the client may pin a stale device. Set the client's

device to match, or set it to "use system default" so it follows Windows.

Symptom D — Headset not recognized / not appearing as a device

Most likely: driver, connection, or firmware — a pure endpoint problem.

1. **Enumerate what Windows sees.** Run **VOIPSUP-Discovery.ps1** — it lists audio endpoints and the

PnP audio devices, including any that are **disabled**, **not present**, or in an **error** state. Also open **Device Manager** (`devmgmt.msc`) → **Audio inputs and outputs** and **Sound, video and game controllers**.

1. **Check for disabled / hidden.** In **Sound Control Panel** → **Recording/Playback**, right-click → **Show

Disabled Devices** / Show Disconnected Devices**. Re-enable the headset if it's disabled.

1. **Check the driver state.** In Device Manager, a yellow bang or "Device cannot start (Code 10/43)"

means a driver/firmware fault. `VOIPSUP-Discovery.ps1` reports PnP device status/error codes so you can spot this without clicking through.

1. **Physical connection.** USB: different port, avoid unpowered hubs, try the headset on another

machine to isolate cable/device vs port. Bluetooth: remove and re-pair; confirm the **Hands-Free** profile is present (not only Stereo/A2DP).

1. **Remediate the driver safely.** `VOIPSUP-Remediation.ps1` can trigger a PnP device rescan and restart

the audio service (default `-WhatIf`). For a corrupt driver, uninstall the device in Device Manager and rescan to reinstall, or deploy the vendor's current headset driver via Intune. **Do not** force-install beta firmware from the field — flag firmware updates for controlled deployment.

1. **Still absent after driver rescan and known-good port/cable?** That's a hardware failure → replace

the headset. Not an escalation to telephony — an endpoint hardware swap.

Symptom E — Jitter / latency / dropped calls (quality degrades or calls drop)

Most likely: network path — measure it, fix what's local, escalate the rest with data.

1. **Measure first.** Run **VOIPSUP-Validation.ps1** to capture **latency, jitter, and packet loss** to

the media-edge target, and **VOIPSUP-Report.ps1** for a readiness summary marked against thresholds.

1. **Connection type.** Move Wi-Fi → wired if at all possible (see Symptom B). On Wi-Fi, check signal,

band (prefer 5 GHz for calls), and roaming between APs mid-call.

1. **Adapter health.** `Get-NetAdapter` for link speed, errors, and driver state; a flapping or

half-duplex NIC causes drops. Update the NIC/Wi-Fi driver via Intune if faulty.

1. **Local saturation and power.** Check background bandwidth use and, on laptops, whether aggressive

power management is throttling the Wi-Fi/USB (Device Manager → adapter → Power Management, or the power plan). Power-saving on the network/USB radios is a real call-drop cause.

1. **VPN / QoS.** As in Symptom B: confirm DSCP marking on the endpoint (`Get-NetQosPolicy`); deploy the

QoS policy if absent. If the endpoint is clean and marked but calls still jitter/drop, the problem is in the **WAN/LAN QoS enforcement or the platform media path → escalate with the numbers.**

1. **Escalation package.** When links 1–4 are clean, attach the **VOIPSUP-Report.ps1** output (device

OK, client version current, latency/jitter/loss with timestamps) to the telephony ticket. That's the difference between a bounced ticket and a solved one.

Symptom F — Softphone client crashes, hangs, or won't launch / no audio in client

Most likely: stale/corrupt client, missing permission, or client using the wrong device.

1. **Version check.** `VOIPSUP-Discovery.ps1` reports the installed client version. Compare against the

minimum supported version from R4. Out-of-date → update via Intune (or the sanctioned installer).

1. **Process health.** `Get-Process` for the client; if it's hung, close it fully (confirm no orphaned

background process remains) and relaunch. Repeated crashes on launch → likely corrupt install or profile.

1. **Microphone permission.** Settings → **Privacy & security** → **Microphone** — the client (and desktop

apps) must be allowed. A denied mic permission presents as "no audio in client" even when the device is perfect.

1. **Device selection inside the client.** Confirm the client's audio device matches the Windows default

communication device (or is set to "system default"). A stale pinned device = no/wrong audio only in the client.

1. **Repair / reset.** `VOIPSUP-Remediation.ps1` can restart the client and the audio service (default

`-WhatIf`). For a corrupt install, reinstall via the sanctioned Intune package. Preserve the client's **log files** (path from R4) before resetting — they're your escalation evidence.

1. **Crash logs & escalation.** If the client crashes with a clean device, current version, granted permission, and correct device selection, capture the client logs and **escalate to the client vendor / telephony team** — in-client crash internals are the vendor's domain, not desktop support's.

Closing every ticket

For any branch, the outcome is one of two clean states:

- **Resolved at the endpoint** — you fixed the device, client, or local network; re-test against the gates in the Testing guide and confirm with the user on a live call.
- **Escalated with evidence** — links 1–4 are provably clean; you hand the telephony team the **VOIPSUP-Report.ps1** output plus what you ruled out. See Best Practices for the escalation template.

Testing

Testing turns "seems fine" into a defensible **PASS/FAIL** result. A VOIP/softphone endpoint is considered healthy only when all three gates pass: the **correct audio device** is active, the **softphone client is healthy**, and the **network is within thresholds**. Run these gates after any remediation and before closing a ticket — and run them again to build the escalation package when the endpoint is clean but calls still fail.

Use the read-only scripts to gather evidence: **VOIPSUP-Validation.ps1** (per-device audio + network) and **VOIPSUP-Report.ps1** (VOIP-readiness/quality summary with PASS/FAIL against thresholds).

Gate 1 — Correct audio device

Check	How to verify	PASS	FAIL
Intended device is present	`VOIPSUP-Discovery.ps1`; Device Manager → Audio inputs/outputs	The intended headset (from R2) appears and is enabled	Missing, disabled, or in an error state
Set as Default Communication Device	`VOIPSUP-Discovery.ps1`; Sound Control Panel (`mmsys.cpl`)	Intended headset is the **Default Communication Device** for both playback and recording	Laptop mic/speaker or another device is the comm default
Mic captures, speaker plays	Recording tab level meter moves on speech; a test tone is audible	Meter moves; tone heard	No meter movement or no output
Not muted / non-zero levels	Recording/Playback → Properties → Levels	Levels above 0 and not muted	Muted or at 0

Gate 1 passes when the intended device is present, enabled, set as the communication default, and actively capturing/playing.

Gate 2 — Softphone client healthy

Check	How to verify	PASS	FAIL
Client version	`VOIPSUP-Discovery.ps1` vs minimum from R4	Installed version ≥ minimum supported	Below minimum / not installed
Process runs cleanly	`Get-Process`; launch and place a test call	Launches and stays running; no crash	Crashes, hangs, or won't launch
Microphone permission	Settings → Privacy & security → Microphone	Client / desktop apps allowed	Blocked
Client uses correct device	Client audio settings	Matches Windows comm default (or "system default")	Pinned to a stale/wrong device

Gate 2 passes when the client is current, stable, permitted to use the mic, and pointed at the correct device.

Gate 3 — Network within thresholds

Run **VOIPSUP-Validation.ps1** against the media-edge target from R3/R5. Compare to your baseline thresholds (set from your platform vendor's endpoint network guidance; defaults below are typical targets — tune to your platform).

Metric	Typical PASS target	FAIL indication
Latency (one-way / round-trip)	Within your platform's published ceiling	Above ceiling → talk-over/echo
Jitter	Below your platform's jitter ceiling	Above ceiling → robotic/choppy audio
Packet loss	Below your platform's loss ceiling	Above ceiling → drop-outs/gaps
Connection type	Wired, or Wi-Fi with strong signal	Weak/congested Wi-Fi
QoS/DSCP marking	Present when required (`Get-NetQosPolicy`)	Absent when platform expects marking

Gate 3 passes when latency, jitter, and loss are all within your documented thresholds and QoS marking is present where required. If Gate 3 fails on WAN/LAN enforcement (endpoint is marked and local path is clean), that is an **escalation to the network/telephony team**, not an endpoint fix.

Overall result

- **PASS (all three gates):** the endpoint is healthy. If calls still fail, escalate to the telephony

team with the **VOIPSUP-Report.ps1** output — the endpoint is provably clean.

- **FAIL (any gate):** remediate that gate (see the Diagnostic Playbook), then re-run all three gates.

Do not close the ticket on a partial pass.

Pilot validation

Before fleet-wide use, run the gates on the pilot group built with **VOIPSUP-Set-ScopeTargeting.ps1**. Confirm the scripts produce consistent PASS/FAIL results across representative hardware (different headsets, docked/undocked, wired/Wi-Fi) and that thresholds are tuned before you expand.

Troubleshooting

This guide covers problems with the **scripts and checks themselves** — not the call-quality symptoms (those are the Diagnostic Playbook). Use it when a script errors, returns nothing, or gives a result you don't trust.

VOIPSUP-Discovery.ps1

"No matching softphone/VOIP client found." The default filter matches common terms (softphone, voip, phone, communicator, calling, meetings). Your client's display name may differ. Re-run with your client's name fragment: `-ClientNameFilter your-client-name`. Confirm the expected name in **Settings** → **Apps** → **Installed apps**.

Audio endpoints list is empty or errors. `Get-PnpDevice -Class AudioEndpoint` requires the PnP subsystem to be responsive. If it errors, the Windows audio stack may be unhealthy — run **VOIPSUP-Remediation.ps1 -RestartAudio -Execute** (needs admin) and re-run Discovery. If still empty, check Device Manager for a missing audio driver.

CSV export fails. The target folder must exist and be writable. Use a path you can write to, e.g. `-CsvPath .\voipsup-discovery.csv`, and don't have the file open in Excel while exporting.

VOIPSUP-Validation.ps1 / VOIPSUP-Report.ps1

"no replies from ..." / Network reachability FAIL. Either the target is wrong/unreachable or ICMP is blocked. Confirm the media-edge target from R3/R5 is correct and reachable (`ping` it manually). If your environment blocks ICMP, measurement against that target won't work — use an approved probe target that responds, and note in the escalation that endpoint-side ICMP measurement was limited by policy.

Latency/jitter look implausibly low or all identical. Some environments answer the first probe from a local cache/gateway. Increase the sample with `-PingCount 30` (or higher) and target the actual media edge, not the default gateway.

Every network gate FAILs but calls are fine (or vice versa). Your thresholds may not match your platform. The defaults are generic starting points — set `-MaxLatencyMs`, `-MaxJitterMs`, `-MaxLossPercent` from your platform vendor's published endpoint network guidance (R5) and re-run. Tune on the pilot before trusting fleet-wide results.

Client version shows but "Version >= minimum" FAILs unexpectedly. Version parsing extracts the numeric portion of the display version. If the client uses a non-standard version string, the comparison may misfire — verify the actual version in **Settings** → **Apps** and, if needed, treat the version gate as informational for that client.

Report shows "client not currently running" as PASS. That's intentional — a not-yet-started client isn't a failure. To test client health, launch the client and place a test call, then re-run so "Process responding" is evaluated.

VOIPSUP-Remediation.ps1

Nothing happens / "No action selected." You must choose at least one action: `-RestartAudio`, `-RescanDevices`, and/or `-RestartClient`. By design the script previews (does nothing) until you also pass `-Execute`.

"requires local administrator rights. Skipping." Restarting the audio service and rescanning devices need admin. Run PowerShell **as administrator** (or via an admin remote-support session) and re-run with `-Execute`.

Client won't relaunch after stop. The script relaunches from the running process's executable path; if that path wasn't available (e.g. the client runs from a per-user store location), relaunch the client manually. This is expected for some packaged clients.

Default communication device isn't changed. In-box Windows cmdlets do not expose setting the Default Communication Device programmatically — the script tells you to set it in **Sound Control Panel (`mmsys.cpl`)** → **Playback/Recording** → **right-click** → **Set as Default Communication Device**. This is intentional (avoids unsupported registry hacks on live user devices).

VOIPSUP-Set-ScopeTargeting.ps1

"Module 'Microsoft.Graph...' is not installed." Install the Graph modules: `Install-Module Microsoft.Graph.Authentication -Scope CurrentUser` and `Install-Module Microsoft.Graph.Groups -Scope CurrentUser`, then re-run.

Graph connection or consent fails. The signed-in account needs the requested scopes (Group.ReadWrite.All, GroupMember.ReadWrite.All, User.Read.All, Device.Read.All) and an admin may need to consent. Confirm the operator has **Intune Administrator** / group-management rights (global requirement G3) and retry.

"User not found" for a valid person. Pass the exact UPN (`user@your-domain`), not a display name or alias. Verify the UPN in Entra → Users.

Members not added even though it "succeeded." Confirm you ran with `-Execute` — in preview mode the script only shows what *would* be added. Re-run with `-Execute` to write the membership.

General

Execution policy blocks the script. Run via `powershell -ExecutionPolicy Bypass -File .\SCRIPT.ps1` (as shown in each script's HOW TO RUN), or set an appropriate policy for the session with `Set-ExecutionPolicy -Scope Process Bypass`.

Read-only vs. change scripts. Discovery, Validation, and Report never change the endpoint. Remediation and Set-ScopeTargeting default to `-WhatIf` (preview) — they only make changes when you add `-Execute`. When in doubt, run without `-Execute` first and read the planned actions.

Best Practices

The point of this solution is not just to fix calls — it is to fix what the endpoint controls and to **escalate everything else with evidence**, so tickets stop ping-ponging between desktop support and the telephony team. These practices keep the method repeatable and make your escalations impossible to bounce back.

Work the tree top-down, every time

Resist the urge to "try things." Walk the root-cause tree in order — **audio device** → **client** → **network/QoS** → **escalate** — and stop at the first broken link. Most call-quality tickets die at the device or client link; you never reach the network. Guessing at the network while the wrong microphone is selected wastes everyone's time and produces a weak escalation.

Define "correct" before you judge "wrong"

You cannot call a device, version, or network condition "wrong" without a documented "right." Keep a baseline (R2/R4/R5): the intended default communication device per role, the minimum supported client version, and the latency/jitter/loss thresholds. Pass/fail gates are only meaningful against thresholds.

Measure the network; never assume it

"It's the network" is the most common — and most often wrong — escalation. Always attach numbers: latency, jitter, and packet loss from **VOIPSUP-Validation.ps1**, with timestamps and connection type (wired/Wi-Fi/VPN). A network claim without measurements is a hunch; a network claim with data is an escalation the network/telephony team can act on immediately.

Keep the three surfaces coherent

The endpoint has three settings that must agree: the **Windows Default Communication Device**, the **client's audio device**, and the **physical headset that's actually connected**. Many "wrong device" and "no audio" tickets are simply these three disagreeing. Set the client to follow the system default where the client supports it, so there's one source of truth.

Prefer wired; treat Wi-Fi and VPN as suspects

For call quality, wired beats Wi-Fi, and direct beats VPN. When quality is poor, moving to wired is the highest-yield single change. Document VPN routing and Wi-Fi conditions when you can't change them — that context belongs in the escalation.

Remediate safely: -WhatIf first, then execute

Every change script (**VOIPSUP-Remediation.ps1**, **VOIPSUP-Set-ScopeTargeting.ps1**) defaults to `-WhatIf`. Read the planned actions, confirm they match your intent, then re-run to execute. Never push firmware or driver changes from the field ad hoc — flag them for controlled Intune deployment and pilot them first. On live user devices, prefer the least-disruptive fix that resolves the gate.

Pilot before fleet-wide

Build a pilot group with **VOIPSUP-Set-ScopeTargeting.ps1** and validate the gates across representative hardware (different headsets, docked/undocked, wired/Wi-Fi) before expanding. Tune thresholds on the pilot so fleet-wide PASS/FAIL results are trustworthy.

Respect the escalation boundary

Desktop support owns the device, the client install, and the local network path. The **dial plan, call routing, media relays/SBC, carrier trunks, server-side codec negotiation, and account/licensing** are the telephony platform's domain. For client-internal crashes and platform console behavior, defer to the vendor's official documentation. Escalating a clean endpoint is the *correct* outcome — not a failure.

Evidence-based escalation to the telephony team

When links 1–4 are provably clean, escalate — but never empty-handed. Attach the **VOIPSUP-Report.ps1** output and a short summary using this template:

Escalation summary — VOIP/softphone endpoint verified clean

- **User / device / date-time of tests:** who, which endpoint, when (with time zone).
- **Symptom:** exact user-reported behavior (no/one-way audio, echo/robotic, jitter, drops, crash) and whether it's every call or intermittent.
- **Gate 1 — Audio device:** PASS. Intended device present, enabled, set as Default Communication Device, capturing/playing. (Attach Discovery output.)
- **Gate 2 — Client:** PASS. Client version (≥ minimum supported), process stable, mic permission granted, client using the correct device.
- **Gate 3 — Network:** measured latency / jitter / packet loss with the values and thresholds, connection type (wired/Wi-Fi/VPN), and whether DSCP marking is present. (Attach Validation/Report output.)
- **Ruled out at the endpoint:** the specific endpoint causes you eliminated (device, driver, permission, client version, local congestion, Wi-Fi, QoS marking).
- **Ask:** what you need the telephony team to check (e.g., media-relay/SBC health, dial-plan routing, WAN/LAN QoS enforcement, server-side codec), given the endpoint is clean.

This turns "please look at this" into "the endpoint is verified clean; here is the data; the problem is on your side of the boundary" — which is what gets call-quality issues actually resolved.

Deliverables Summary

This solution is a complete, reusable toolkit for the **endpoint side** of enterprise VOIP/softphone support. It pairs a symptom-driven method with five PowerShell scripts so desktop support can resolve the call-quality tickets it controls and escalate the rest with evidence.

Documents

Document	What it gives you
01 — Solution Overview	The softphone/VOIP endpoint stack (audio device → client → network/QoS → service), what the endpoint owns vs. the telephony platform, and the root-cause tree.
02 — Requirements	Each prerequisite (endpoint access, audio-device inventory, network/QoS visibility, softphone client details, known-good baseline) with Description / Why / Verify / Implement-if-missing.
03 — Diagnostic Playbook	The symptom tree — no/one-way audio, echo/robotic, wrong default device, headset not recognized, jitter/latency, client crash — with exact endpoint-side checks per branch.
05 — Testing	Pass/fail gates: correct device, client healthy, network within thresholds.
06 — Troubleshooting	Fixes for the scripts and the checks themselves when they don't behave.
07 — Best Practices	Evidence-based escalation to the telephony team, including the escalation template.
Global Requirements (Infrastructure)	The category-level prerequisites shared by all Infrastructure solutions.

Scripts

All scripts share a standard header (PURPOSE / HOW TO RUN / EXPECTED RESULTS). Discovery, Validation, and Report are **read-only**. Remediation and Set-ScopeTargeting **default to** `-WhatIf` — they show planned actions until you explicitly execute.

Script	Role	Mode
VOIPSUP-Discovery.ps1	Inventory audio input/output devices (defaults, PnP status) and the installed softphone client + version.	Read-only
VOIPSUP-Validation.ps1	Per-device audio checks plus network measurement (latency, jitter, packet loss) against thresholds.	Read-only
VOIPSUP-Report.ps1	VOIP-readiness / call-quality summary marking each gate PASS/FAIL — the escalation package.	Read-only

Script	Role	Mode
VOIPSUP-Remediation.ps1	Safe endpoint fixes: set the Default Communication Device, restart the audio service, rescan devices, restart the client.	`-WhatIf` default
VOIPSUP-Set-ScopeTargeting.ps1	Build/assign the pilot security group via Microsoft Graph so you validate on a controlled set first.	`-WhatIf` default

How they fit the method

Assess with **Discovery** → validate the endpoint and network with **Validation** → summarize gates with **Report** → apply safe fixes with **Remediation** → pilot on a controlled group with **Set-ScopeTargeting**, then expand. Resolve at the endpoint, or escalate with the Report output.

Outcome

Call-quality and softphone tickets resolved at the endpoint — or escalated to the telephony team with hard evidence — using a repeatable method and scripts your support team keeps and reuses.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

VOIPSUP-Discovery.ps1

Purpose: Collects the read-only facts a technician needs before troubleshooting a VOIP/softphone call-quality ticket: every audio input/output endpoint, which device is the current default communication device, PnP audio device status/error codes, and the installed softphone/VOIP client with its version. Change nothing; this only reports what the endpoint currently has. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Discovery.ps1 -ClientNameFilter softphone-client-name -CsvPath .\voipsup-discovery.csv EXPECTED RESULTS A console table of audio playback/recording devices (with the default-communication device flagged), a list of PnP audio devices with status, and the detected softphone client(s) and version(s). Optional CSV export. No PASS/FAIL — this is inventory, not a gate. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Discovery.ps1 -ClientNameFilter softphone-client-name -CsvPath .\voipsup-discovery.csv EXPECTED RESULTS A console table of audio playback/recording devices (with the default-communication device flagged), a list of PnP audio devices with status, and the detected softphone client(s) and version(s). Optional CSV export. No PASS/FAIL — this is inventory, not a gate. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

VOIPSUP-Remediation.ps1

Purpose: Applies the low-risk endpoint remediations that resolve most VOIP/softphone tickets: set the default communication device to the intended headset, restart the Windows audio service, rescan for PnP devices (headset not recognized), and restart the softphone client. Every action defaults to preview only (-WhatIf) so you review before anything changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 (preview / -WhatIf) powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 -RestartAudio -RescanDevices -Execute powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 -RestartClient -ClientProcessName softphone-client-name -Execute EXPECTED RESULTS In preview mode: a list of the actions that WOULD run, no changes made. With -Execute: each selected action runs and reports success/failure. Some actions (audio service, device rescan) require local administrator rights. .NOTES White-label — the repeatable way. Read-only for discovery/validation. Remediation defaults to -WhatIf; pass -Execute (or -WhatIf:$false) to apply. #>

How to run: powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 (preview / -WhatIf) powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 -RestartAudio -RescanDevices -Execute powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Remediation.ps1 -RestartClient -ClientProcessName softphone-client-name -Execute EXPECTED RESULTS In preview mode: a list of the actions that WOULD run, no changes made. With -Execute: each selected action runs and reports success/failure. Some actions (audio service, device rescan) require local administrator rights. .NOTES White-label — the repeatable way. Read-only for discovery/validation. Remediation defaults to -WhatIf; pass -Execute (or -WhatIf:$false) to apply. #>

VOIPSUP-Report.ps1

Purpose: Read-only reporting that rolls the three call-quality gates into one summary: correct/healthy audio device, healthy softphone client (version vs a supplied minimum), and network within thresholds. Emits a console summary and an optional HTML/CSV report you can attach to a telephony-team escalation. Changes nothing on the endpoint. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Report.ps1 -Target media-edge-host-or-ip -MinClientVersion 1.0.0.0 -ClientNameFilter softphone-client-name powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Report.ps1 -Target media-edge-host-or-ip -HtmlPath .\voipsup-report.html EXPECTED RESULTS A per-gate PASS/FAIL summary (device / client / network) and an OVERALL readiness verdict. Optional HTML and CSV files for the escalation package. All-PASS with failing calls means the endpoint is clean — escalate. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Report.ps1 -Target media-edge-host-or-ip -MinClientVersion 1.0.0.0 -ClientNameFilter softphone-client-name powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Report.ps1 -Target media-edge-host-or-ip -HtmlPath .\voipsup-report.html EXPECTED RESULTS A per-gate PASS/FAIL summary (device / client / network) and an OVERALL readiness verdict. Optional HTML and CSV files for the escalation package. All-PASS with failing calls means the endpoint is clean — escalate. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

VOIPSUP-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) a Microsoft Entra security group for the VOIP/softphone endpoint-support pilot and adds the specified members (users or devices) via Microsoft Graph, so you validate the method and scripts on a controlled set before fleet-wide use. Defaults to preview only (-WhatIf) so you review the planned group and membership changes before anything is written to Entra. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Set-ScopeTargeting.ps1 -GroupName VOIPSUP-Pilot -MemberUpns user1@your-domain,user2@your-domain powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Set-ScopeTargeting.ps1 -GroupName VOIPSUP-Pilot -MemberUpns user1@your-domain -Execute (Requires the Microsoft.Graph modules and Group.ReadWrite.All / GroupMember.ReadWrite.All consent.) EXPECTED RESULTS In preview mode: the group that WOULD be created/reused and the members that WOULD be added, no changes made. With -Execute: the group is created if absent and members are added, with a confirmation line per member. .NOTES White-label — the repeatable way. Read-only for discovery/validation. Scope-targeting defaults to -WhatIf; pass -Execute (or -WhatIf:$false) to apply. #>

How to run: powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Set-ScopeTargeting.ps1 -GroupName VOIPSUP-Pilot -MemberUpns user1@your-domain,user2@your-domain powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Set-ScopeTargeting.ps1 -GroupName VOIPSUP-Pilot -MemberUpns user1@your-domain -Execute (Requires the Microsoft.Graph modules and Group.ReadWrite.All / GroupMember.ReadWrite.All consent.) EXPECTED RESULTS In preview mode: the group that WOULD be created/reused and the members that WOULD be added, no changes made. With -Execute: the group is created if absent and members are added, with a confirmation line per member. .NOTES White-label — the repeatable way. Read-only for discovery/validation. Scope-targeting defaults to -WhatIf; pass -Execute (or -WhatIf:$false) to apply. #>

VOIPSUP-Validation.ps1

Purpose: Read-only validation for a VOIP/softphone endpoint. Confirms audio endpoints are present and healthy, reports the active network adapter and connection type, and measures latency, jitter, and packet

loss to a media-edge target so you can judge call quality against thresholds. Marks each measured item PASS/FAIL. Changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Validation.ps1 -Target media-edge-host-or-ip powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Validation.ps1 -Target media-edge-host-or-ip -MaxLatencyMs 150 -MaxJitterMs 30 -MaxLossPercent 1 -PingCount 30 EXPECTED RESULTS A device-health line per audio endpoint, the active adapter/connection type, and measured latency/jitter/loss with PASS or FAIL against the supplied thresholds. Use with VOIPSUP-Report.ps1 to build the escalation package. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Validation.ps1 -Target media-edge-host-or-ip powershell -ExecutionPolicy Bypass -File .\VOIPSUP-Validation.ps1 -Target media-edge-host-or-ip -MaxLatencyMs 150 -MaxJitterMs 30 -MaxLossPercent 1 -PingCount 30 EXPECTED RESULTS A device-health line per audio endpoint, the active adapter/connection type, and measured latency/jitter/loss with PASS or FAIL against the supplied thresholds. Use with VOIPSUP-Report.ps1 to build the escalation package. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>